

Lab 4 Report – HLS Optimization of FIR Filter

EE19B108

Vikas Raj

Objective

The goal of this experiment was to study how HLS directives and clock constraints affect the synthesized hardware architecture, scheduling, and resource usage of a FIR filter implemented in Vitis HLS. The task was to obtain a valid synthesis solution while limiting the number of floating-point arithmetic units used by the design.

HLS Settings Used

The following directives were used in the implementation:

```
#pragma HLS PIPELINE II=1
#pragma HLS ALLOCATION operation instances=fmul limit=1
#pragma HLS ALLOCATION operation instances=fadd limit=2
#pragma HLS INTERFACE mode=axis port=x_in
#pragma HLS INTERFACE mode=axis port=return
#pragma HLS INTERFACE mode=s_axilite port=return
```

Clock constraint used:

Target Clock Period = 50 ns

Explanation of the Design Settings

1. Loop Pipelining

```
#pragma HLS PIPELINE II=1
```

This directive pipelines the loop so that a new input sample can begin processing every clock cycle.

An Initiation Interval (II) of 1 means the system can accept continuous streaming input without waiting for the previous iteration to complete. This significantly improves throughput and is commonly used in streaming DSP designs such as FIR filters.

2. Limiting Multipliers

```
#pragma HLS ALLOCATION operation instances=fmul limit=2
```

This directive restricts the design to only 2 floating-point multipliers.

Instead of generating many parallel multipliers, the scheduler must reuse the same multiplier across multiple cycles. This reduces DSP usage and hardware cost, but it also forces operations to be scheduled sequentially.

3. Limiting Adders

```
#pragma HLS ALLOCATION operation instances=fadd limit=3
```

This directive limits the number of floating-point adders to 3.

Multiple addition operations must therefore share these adders, which again reduces hardware usage but increases scheduling complexity.

4. AXI Stream Interfaces

```
#pragma HLS INTERFACE mode=axis port=x_in  
#pragma HLS INTERFACE mode=axis port=return
```

These directives configure the module as a streaming accelerator using AXI-Stream interfaces.

- `x_in` receives streaming input samples
- `return` sends output samples

This interface type is commonly used in DSP pipelines where data arrives continuously.

5. AXI Lite Control Interface

```
#pragma HLS INTERFACE mode=s_axilite port=return
```

This directive adds a control interface using AXI-Lite.

It allows the processor to start, stop, and configure the accelerator when the design is packaged as an IP core.

Importance of the 50 ns Clock Constraint

The design uses restricted arithmetic resources:

- 2 floating-point multiplier
- 3 floating-point adders

Floating-point operations require multiple clock cycles to complete. When the clock target was set to 10 ns, the HLS scheduler could not properly schedule all operations while respecting the resource limits.

By relaxing the clock constraint to 50 ns, the scheduler had enough time to:

- reuse arithmetic units
- sequence operations across cycles
- satisfy timing requirements

Thus, the design could be successfully synthesized while maintaining the resource limits.

Key Findings

1. Resource allocation directives strongly influence the generated hardware.
Limiting arithmetic units forces the HLS tool to reuse hardware instead of creating parallel units.
2. Clock constraints affect the scheduler's ability to allocate operations.
A very tight clock constraint can prevent successful scheduling when resources are limited.

3. Relaxing the clock enables resource sharing.
With a 50 ns clock period, the tool successfully scheduled all operations using only 2 multipliers and 3 adders.
 4. There is a trade-off between performance and resource usage.
Fewer hardware units reduce FPGA resource consumption but increase operation scheduling across cycles.
-

Conclusion

This experiment demonstrates how HLS directives and timing constraints determine the final hardware architecture. By applying pipelining, limiting arithmetic resources, and using AXI streaming interfaces, the FIR filter could be synthesized successfully with controlled hardware usage. A 50 ns clock constraint was necessary to allow the scheduler to reuse the limited floating-point arithmetic units and produce a valid synthesis solution.